

AI Systems Performance Engineering

Chapter 1 — Introduction and AI System Overview

Kaige Yang
Senior Applied Scientist
AWS

Internal Training for Applied Scientists on AI Infrastructure

August 13, 2026

About This Book

The definitive guide to maximizing efficiency across every layer of AI infrastructure

In today's era of ever-growing generative models, **AI Systems Performance Engineering** gives engineers, researchers, and developers a **hands-on** set of actionable optimization strategies.

- **Co-optimize hardware, software, and algorithms** to build resilient, scalable, and cost-effective AI systems.
- Excel at both **training** and **inference**.
- Turn complex AI systems into **streamlined, high-impact** solutions.

*Authored by **Chris Fregly** — a performance-focused engineering and product leader.*

What's Inside

- Step-by-step methodologies for fine-tuning **GPU CUDA kernels**, **PyTorch**-based algorithms, and **multinode** training & inference systems.
- Master scaling **GPU clusters** for high-performance, distributed training jobs and inference servers.
- Ends with a **175+-item checklist** of proven, ready-to-use optimizations.

What you'll be able to do

- Codesign hardware, software & algorithms for **maximum throughput and cost savings**.
- Implement cutting-edge **inference strategies** that cut latency and boost throughput.
- Use industry-leading **scalability tools and frameworks**.
- **Profile, diagnose, and eliminate** bottlenecks across complex AI pipelines.
- Integrate **full-stack optimization** for robust, reliable AI system performance.

About the Author — Chris Fregly

Background

An **AI product and performance engineering leader** with experience at **AWS**, **Databricks**, and **Netflix**.

- Coauthored *Data Science on AWS* and *Generative AI on AWS* (O'Reilly).
- Created an O'Reilly course on **optimizing AI performance with NVIDIA GPUs**.
- Founded the global **AI Performance Engineering meetup**.

Focus today

- GPU / CUDA & PyTorch performance tuning
- Distributed training & inference optimization
- Full-stack AI systems performance engineering

This deck distills that hands-on, production-tested perspective.

Agenda

- ① Why This Matters
- ② The AI Systems Performance Engineer
- ③ Case Study: DeepSeek on Constrained Hardware
- ④ Toward 100-Trillion-Parameter Models
- ⑤ NVIDIA's AI Supercomputer in a Rack
- ⑥ Two Core Principles
- ⑦ Book Roadmap & Key Takeaways

The DeepSeek Wake-Up Call (late 2024)

- A small Chinese startup trained a **frontier LLM** *without* access to top-tier NVIDIA GPUs (Blackwell / Hopper).
- Export restrictions $\Rightarrow$ forced onto the **H800** (export-compliant, reduced NVLink & FP64).
- Response: **custom CUDA kernels**, model distillation, and treating **communication bandwidth as a scarce resource** — optimizing every byte.
- Result: **DeepSeek-R1** reached reasoning quality near frontier models trained on far more capable hardware.^[1]

The takeaway

Practitioners skilled in **AI systems performance engineering** get the most out of available hardware — *no matter the constraints*.

Brute Force vs. Skillful Engineering

Brute force (US/EU frontier labs)

- Ever-larger clusters and models
- Millions → billions → trillions of params
- GPT-4 (2023): ~\$100M to train
- Gemini Ultra (2023): ~\$191M^[10]

Efficiency (DeepSeek)

- DeepSeek-R1 claim: <\$6M compute^[2]
- An **order of magnitude** cheaper
- Matched rival model performance
- Shocked markets: NVIDIA stock -17% in a day

At ultrascale, every bit of performance = millions or billions of dollars.

Who Is the AI Systems Performance Engineer?

A specialized role optimizing AI models *and* the systems they run on — making training & inference **fast, cost-efficient, reliable**.

- Blends **hardware + software + algorithms** — codesigned, not siloed.
- Must know: low-level OS, memory hierarchies, networking, Python/C++, PyTorch, Triton, CUDA.
- A typical day: GPU kernel efficiency → OS thread scheduling → memory access patterns → network throughput → distributed-training debugging.

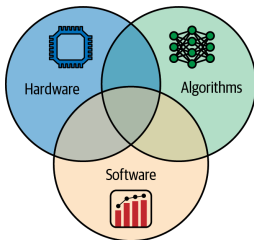


Figure 1-1. Codesigning hardware, software, and algorithms.

Key Tools & Terms (used on the next slide)

Profiling tools

- **Nsight Systems** — system-wide timeline profiler.
Plain English: a stopwatch for the whole machine — shows where time is wasted waiting.
- **Nsight Compute** — single-kernel deep profiler.
Plain English: a microscope on one GPU operation — why is this step slow?
- **PyTorch Profiler** — in-framework profiler.
Plain English: shows which lines of your PyTorch code cost the most GPU time.

Scaling & resource terms

- **NCCL** — collective comm library (**training**).
Plain English: how many GPUs quickly share and sync their results.
- **NIXL** — inference transfer library (**inference**).
Plain English: fast delivery service that shuttles data between GPUs during inference.
- **MIG** — Multi-Instance GPU.
Plain English: slice one big GPU into smaller ones so small jobs don't waste it.

Profilers tell you **where** time goes; NCCL/NIXL move data; MIG right-sizes the GPU.

Core Responsibilities

Benchmarking & Profiling

Measure latency, throughput, memory. Iterate with **Nsight Systems**, **Nsight Compute**, and the **PyTorch profiler**. Automate perf tests to catch regressions early.

Scaling Training & Inference

8 → 80,000 GPUs with minimal overhead. **NCCL** for training collectives (all-reduce/all-gather/all-to-all); **NIXL** for inference data movement. Data / tensor / pipeline / expert parallelism.

Managing Resources

Feed GPUs at full throttle, pin threads, cut context switches, avoid **OOM**. Partition with **MIG** when full GPU isn't needed.

Collaboration + Reproducibility

Sits across researchers, DevOps, infra. **Measure everything, trust data not assumptions**. Publish so others reproduce and build on the work.

Transparency: DeepSeek's Open-Source Week (Feb 2025)

- **FlashMLA** — optimized attention kernel (CUDA C++)
- **DeepGEMM** — FP8-optimized matmul, beats many vendor kernels
- **DeepEP** — communication library for MoE models
- **EPLB** — redundant-expert load balancer (duplicate hot experts)
- **DualPipe** — bidirectional pipeline parallelism, overlaps compute/comm
- **3FS** — Fire-Flyer distributed filesystem (every layer must be optimized)^[3]

Why open sourcing matters

Releasing production-tested code let others **benchmark, reproduce, and build on** the results — turning efficiency claims into verifiable, community-usable baselines.

Open-Source Week — What Each Tool Does (Plain English)

- **FlashMLA** — *Attention, done fast.*

A hand-tuned kernel that runs the model's attention step quickly while using little memory.

- **DeepGEMM** — *Faster multiplication.*

The core matrix-multiply behind every layer, optimized in low-precision FP8 — often beating the vendor's own.

- **DeepEP** — *Talking between experts.*

Moves tokens to the right experts across GPUs efficiently — the messaging system for MoE models.

- **EPLB** — *Balancing the load.*

Duplicates the popular experts so no single GPU gets overwhelmed while others sit idle.

- **DualPipe** — *Never wait idle.*

Overlaps computation with communication so GPUs keep working instead of waiting for data.

- **3FS** — *A fast filesystem.*

A distributed storage system that feeds data to GPUs fast enough to keep them busy.

*Takeaway: they optimized **every layer** — math, memory, networking, and storage.*

MLPerf v5.0 (2025) — Blackwell GB200 NVL72 vs. Hopper (per GPU)

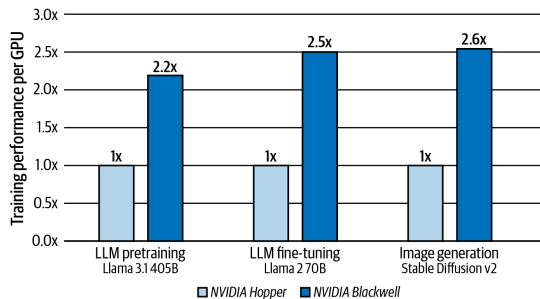


Figure 1-2. Training v5.0 — up to $2.6\times$.^[4]

Y-axis: **relative per-GPU training performance**, normalized to Hopper = $1\times$ (from MLPerf time-to-train). Higher = faster.

In plain English: if one **Hopper** GPU trains a fixed chunk in **26** min, one **Blackwell** GPU does the same in $\sim$ **10** min — **$2.6\times$** the work in the same time.

Caveat: per-GPU is a component-level indicator — compare system-level, end-to-end throughput.

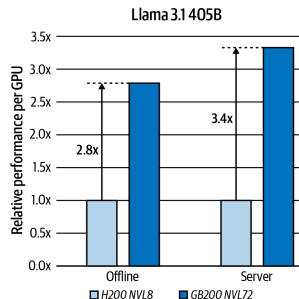


Figure 1-3. Inference v5.0 — up to $3.4\times$.^[5]

Y-axis: **relative per-GPU inference throughput** (tokens/s), normalized to Hopper = $1\times$. Higher = more work/GPU.

DeepSeek Scales to $\sim 680\text{B}$ Parameters Under Restrictions

The hardware gap (per GPU):

	H100	H800
NVLink BW	$\sim 900\text{ GB/s}$	$\sim 400\text{ GB/s}$
HBM BW	$\sim 3.35\text{ TB/s}$	$\sim 3.35\text{ TB/s}$ (same)
FP64	full	reduced (throttled)

*FP64 = double precision, used only in HPC/scientific computing — **not** in LLM training (BF16/FP8). Throttling it doesn't slow DeepSeek; the real cut was **NVLink** (halved), not memory bandwidth.*

DeepSeek-V3: a sparse MoE^[2]

- $\sim 680\text{B}$ total params
- Only $\sim 37\text{B}$ **active** per token
- 1 shared + 8 routed experts (of 256) = 9 active

Smart software & algorithm design overcame brute-force hardware limits.

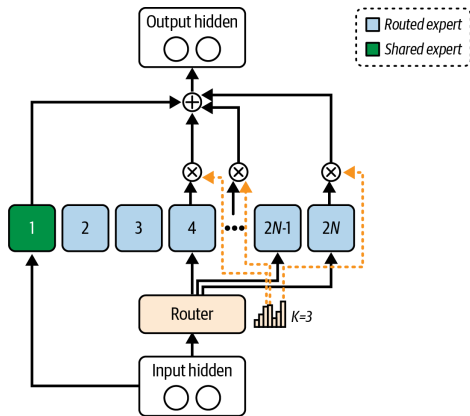
How they beat the constraints

- **DualPipe**: overlap compute + comm to mask slow interconnect
- Custom CUDA kernels bypass default NCCL collectives
- Keep GPUs busy despite reduced bandwidth

DeepSeek-R1

Built on V3; **“cold start”** + reinforcement learning instead of costly human feedback $\Rightarrow$ chain-of-thought reasoning, lower cost.

How MoE Routing Works (DeepSeek-V3)



- A **router** scores experts per token and selects the top- K .
- A **shared expert** always runs; **routed experts** are chosen per token.
- Only the selected experts' outputs are combined $\Rightarrow$ compute stays bounded even as total experts grow.
- In V3: 1 shared + 8 routed (of 256) = 9 active $\Rightarrow \sim 37\text{B}$ of $\sim 680\text{B}$ params active per token.

Figure 1-4. DeepSeek-V3's expert routing.^[2]

What Is FLOPS? (and How to Compute It)

Definition

FLOPS = Floating-Point Operations Per Second — the *rate* of decimal math (adds/multiplies) a chip can do. More FLOPS $\Rightarrow$ more compute.

Hardware peak

- $\text{peak} = (\# \text{ math units}) \times (\text{ops/unit/clock}) \times (\text{clock speed})$.
- A fused multiply-add (FMA) counts as **2** ops.
- **Lower precision $\Rightarrow$ more FLOPS**: each step $\text{FP16} \rightarrow \text{FP8} \rightarrow \text{FP4}$ roughly **doubles** throughput; sparsity adds $\sim 2\times$.

Units: TFLOPS = 10^{12} , PFLOPS = 10^{15} , exaFLOPS = 10^{18} ops/s.

Counting a workload's FLOPs

- Matmul ($M \times K$ by $K \times N$) = $2MNK$ FLOPs.
- Training a dense model $\approx 6 \times \text{params} \times \text{tokens}$.
- Inference (forward) $\approx 2 \times \text{params} \times \text{tokens}$.

Per-GPU Peak FLOPS (dense Tensor Core, per GPU)

GPU	BF16/FP16	FP8	FP4	Notes
H20	~0.15 PF	~0.30 PF	—	cut-down; big memory, weak compute
H100	~0.99 PF	~1.98 PF	—	Hopper baseline
H800	~0.99 PF	~1.98 PF	—	=H100 compute; FP64 & NVLink cut
H200	~0.99 PF	~1.98 PF	—	=H100 compute; more/faster HBM
GB200	~2.5 PF	~5 PF	~10 PF	per Blackwell (B200) GPU
GB300	~2.5 PF	~5 PF	~15 PF	Blackwell Ultra (B300); 1.5× FP4

Dense Tensor-Core figures, *per GPU*. Multiply by ~2× for NVIDIA's "with sparsity" headline numbers. Hopper (H20/H100/H800/H200) has **no FP4**; FP4 is new in Blackwell. GB200/GB300 are Grace+Blackwell superchips = 2 **GPUs** each. Same rule everywhere: lower precision + sparsity = more FLOPS.

The 100-Trillion-Parameter Aspiration

Compared to the ~ 100 trillion synaptic connections in the human neocortex.

Why brute force fails

- Dense 100T model on ~ 29 T tokens $\approx 1.2 \times 10^{29}$ FLOPS.
- Memory: ~ 182 TB at 16-bit precision ($100\text{T} \times 16 \text{ bits} \div 8$).
- That's $\sim 1000\times$ a single B200's 192 GB (180 usable).
- Just to *load* weights: ~ 1000 B200s (~ 125 nodes) or ~ 700 B300s (~ 86 nodes) — before activations & optimizer states.

The answer: sparsity (MoE)

Activate only a subset of “experts” per token $\Rightarrow$ FLOPS/token stays roughly constant as total params grow. Lower inference latency, too. (e.g., *Google Switch Transformer, 1.6T params, trained 7 $\times$ faster than dense.*)^[9]

Why We Need a Rack-Scale Machine

The problem the 100T numbers create

A model this size can't fit on one GPU — it must be **split across ~1000 GPUs** just to hold the weights (more for activations, optimizer states, KV cache).

Splitting $\Rightarrow$ constant talking

- Every step, GPUs must **exchange data** (all-reduce, all-to-all for MoE).
- In a normal cluster, GPUs in different nodes talk over **slow** network links.
- If GPUs **wait on the network**, expensive compute sits idle.

***What “rack” means:** the tall $\sim 42U$ equipment cabinet in a data center that holds the servers. **Rack-scale** = the **whole rack** (all 72 GPUs, wired together) behaves as **one machine** — not a pile of separate servers. The bottleneck at scale isn't FLOPS — it's **moving data between GPUs**. That's what a rack-scale machine solves.*

What we actually need

- Many GPUs that behave like **one giant GPU**.
- A **huge, uniform, high-bandwidth** interconnect so data moves almost as fast between GPUs as inside one.
- Keep every GPU **fed and busy** — the whole point of the next slide.

GB200 NVL72 — “AI Supercomputer in a Rack”

Composition

- 36 Grace Blackwell Superchips
- Each = 1 Grace CPU (72 cores) + 2 Blackwell GPUs
- $\Rightarrow$ 36 CPUs + **72 GPUs** (the “72” in NVL72)
- Connected via **NVLink + NVSwitch**

Interconnect

- NVLink 5: **1.8 TB/s** bidirectional per GPU
- Rack aggregate: **~ 130 TB/s** GPU $\leftrightarrow$ GPU

Rack-level performance

- **~ 1.44 exaFLOPS** FP4 (2:1 sparsity)^[12]
- **~ 720 petaFLOPS** FP8 (2:1 sparsity)
- ~ 13.5 TB HBM3e; ~ 30 TB with Grace memory
- ~ 120 – 132 kW rack power

Roadmap continues the doubling trend: **Vera Rubin** (2026), **Feynman** (2028), and beyond.

Principle 1 — Mechanical Sympathy 机械感知力 (HW/SW Codesign)

Definition

Writing software **deeply aware of the hardware** it runs on (coined by Martin Thompson; analogy to F1 driver Jackie Stewart).^[11]

- **FlashAttention**: “tiles” computation to minimize GPU memory reads/writes $\Rightarrow$ **2–4 $\times$** speedup on long sequences, less memory.^[6]

Plain English: instead of writing the huge attention scratch-work out to slow memory, keep it in fast on-chip memory in small tiles — same answer, far less waiting.

- **DeepSeek MLA** (Multi-Head Latent Attention): restructures attention for the memory hierarchy & Tensor Cores — beat FlashAttention on H800.^[7]

Plain English: shrink the attention “memory” (KV cache) into a small compressed form, so less data moves — a big win on the bandwidth-starved H800.

- Virtuous cycle: transformers + FP8/FP4 $\Rightarrow$ Transformer Engine & reduced-precision Tensor Cores $\Rightarrow$ new algorithms $\Rightarrow$ new hardware. . .

Plain English: new algorithms inspire new hardware, and new hardware inspires new algorithms — each round makes the other better, spinning faster over time.

This entire book is a study in mechanical sympathy.

Principle 2 — Measure “Goodput” (Useful Throughput)

Goodput

Useful work completed per unit time — discounting stalls, idle compute, restarts. Normalize by peak $\Rightarrow$ % efficiency. (Meta: “effective training-time ratio.”)^[8]

Worked example

8 GPUs process 100,000 tokens in 10 s $\Rightarrow$ **10,000** tok/s goodput.

Peak = 1,500 tok/s/GPU $\times$ 8 = 12,000 tok/s. Efficiency = $10,000/12,000 = 83.3\%$.

- Meta found headline util can be **100%** while **70–75%** of compute is lost to comm delays, poor parallelism, data stalls, failure recovery.
- Fixes: caching / async prefetch for data stalls; **overlap** gradient compute with cross-GPU communication.
- A **20%** efficiency gain can save *millions* at scale.

A Close Cousin: Model FLOPs Utilization (MFU)

Definition

MFU = fraction of the hardware's **peak FLOPS** your model turns into useful math:
with useful work $\approx 6 \times \text{params} \times \text{tokens/s}$ (training).

$$\frac{\text{useful FLOPS}}{\text{peak FLOPS}},$$

Worked example

70B model at **1,000** tok/s/GPU on H100.

Useful = $6 \times 70\text{B} \times 1,000 = \mathbf{0.42 \text{ PF}}$.

H100 peak (BF16) $\approx \mathbf{0.99 \text{ PF}}$.

$\Rightarrow \mathbf{MFU} = 0.42/0.99 \approx 42\%$.

Real-world anchors

- GPT-3 (2020): $\sim \mathbf{21\%}$
- PaLM (2022): $\sim \mathbf{46\%}$ (excellent for its era)
- Well-tuned modern LLM: **35–55%**
- $> \mathbf{60\%}$ is rare — needs FP8 + heavy compute/comm overlap

*Like goodput, MFU exposes waste: **nvidia-smi** can read **100%** “utilization” while MFU is only **30%** — busy $\neq$ useful.*

Roadmap of the Book

- **Ch. 2:** NVIDIA AI hardware — GB200/GB300 NVL72.
- **Ch. 3–5:** OS, networking, storage tuning (pinning, Docker, K8s, I/O).
- **Ch. 6–12:** CUDA fundamentals & kernel optimizations (FlashAttention, MLA).
- **Ch. 13–14:** PyTorch, `torch.compile`, Triton; DP/FSDP/TP/PP/CP/MoE; activation checkpointing, optimizer sharding, CPU offload.
- **Ch. 15–19:** Inference & agentic systems — disaggregated prefill/decode, vLLM/SGLang/TensorRT-LLM, NVIDIA Dynamo + NIXL, quantization, speculative decoding.
- **Ch. 20:** AI-assisted optimization; toward 100T-param systems.
- **Appendix:** performance & cost-saving checklist.

Method: hands-on, empirical — profile, hypothesize, measure, verify.

Key Takeaways

- ① **Measure goodput**, not raw FLOPS or utilization — useful work / time.
- ② **Skillful engineering > brute-force spending** — DeepSeek on H800s.
- ③ **Order-of-magnitude impact from incremental gains** — small % = \$millions.
- ④ **Profile-driven mindset** — let data find the true bottleneck, then target it.
- ⑤ **Holistic view** — HW/SW/algorithm codesign; any weak layer bottlenecks all.
- ⑥ **Stay current** — unified memory, faster interconnects, new precision formats, MLPerf.

Mantra

Mechanical sympathy — understand the machinery deeply, and exploit the entire stack to make the impossible both *possible* and *affordable*.

Questions & Discussion

How can we raise *goodput* on our own workloads?

References & Further Reading I

- ① DeepSeek-AI. *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via RL*. arXiv:2501.12948 (2025).
- ② DeepSeek-AI. *DeepSeek-V3 Technical Report*. arXiv:2412.19437 (2024). 671B/37B-active MoE + MLA; DualPipe; 1 shared + 8-of-256 experts; 2.788M H800 GPU-hours $\approx$ \$5.6M compute.
- ③ DeepSeek-AI. *Open-Infra-Index* — Open-Source Week, Feb 2025. github.com/deepseek-ai/open-infra-index.
- ④ NVIDIA. *Blackwell Delivers up to 2.6 $\times$ in MLPerf Training v5.0* (2025). NVIDIA Developer Blog; MLCommons v5.0 results.
- ⑤ NVIDIA. *Blackwell Delivers Massive Leaps in MLPerf Inference v5.0* (2025). NVIDIA Developer Blog (3.4 $\times$ /GPU; up to 30 $\times$ rack-scale).
- ⑥ Dao et al. *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness*. arXiv:2205.14135 (NeurIPS 2022).
- ⑦ DeepSeek-AI. *DeepSeek-V2* — introduces Multi-Head Latent Attention (MLA). arXiv:2405.04434 (2024).
- ⑧ Kokolis et al. (Meta). *Revisiting Reliability in Large-Scale ML Research Clusters* — Effective Training-Time Ratio. arXiv:2410.21680 (2024).
- ⑨ Fedus, Zoph, Shazeer. *Switch Transformers*. arXiv:2101.03961 (2021; JMLR 2022).
- ⑩ Stanford HAI. *2024 AI Index Report* — frontier training costs (Gemini Ultra $\sim$ \$191M; GPT-4 $\sim$ \$78M per AI Index). hai.stanford.edu/ai-index/2024-ai-index-report.
- ⑪ Thompson, M. *Why Mechanical Sympathy* (2011). mechanical-sympathy.blogspot.com.
- ⑫ NVIDIA. *GB200 NVL72 product page*. nvidia.com/.../gb200-nvl72.